

Cell 6 — Shared Utility Functions (Notebook cell index: 12)

Globals `all_results` / `tuning_logs`, plus three functions: `get_preprocessed_splits`, `evaluate_and_store`, `add_tuning_log`.

Granular line-by-line breakdown

Result containers

```
all_results = []
tuning_logs = []
```

- `all_results` — Empty **list** that will hold **dictionaries** (one per final test evaluation).
 - `tuning_logs` — Empty **list** for **validation** tuning rows.
-

`get_preprocessed_splits`

```
def get_preprocessed_splits(prepare_fn):
```

- `prepare_fn` — Parameter: **one** of `preprocess_none`, `preprocess_extreme`, or `preprocess_optimum` (callable).

```
    X_train_text = train_part['text'].apply(prepare_fn).astype(str).values
```

- `train_part` — DataFrame from split cell (training fold).
- `train_part['text']` — Series of raw headline strings.
- `.apply(prepare_fn)` — Calls `prepare_fn` on **each** headline; result Series has cleaned/transformed strings.
- `.astype(str)` — Forces dtype to string (guarantees object dtype for downstream APIs).
- `.values` — NumPy **ndarray** of shape `(n_train,)` — plain array without index.

```
    X_val_text = val_part['text'].apply(prepare_fn).astype(str).values
    X_test_text = work_test['text'].apply(prepare_fn).astype(str).values
```

- Same pattern for validation rows `val_part` and held-out `work_test`.

```
    return X_train_text, X_val_text, X_test_text
```

- Returns **three** arrays (tuple unpacking where called).

`evaluate_and_store` — metrics

```
def evaluate_and_store(y_true, y_pred, prep_name, rep_name, model_name):
```

- `y_true` — Ground-truth integer labels (same encoding as `y_test`).
- `y_pred` — Predicted integer labels from a classifier.
- `prep_name` — String like 'none', 'extreme', 'optimum'.
- `rep_name` — 'TF-IDF' or 'Skip-gram'.
- `model_name` — e.g. 'LogisticRegression', 'DeepNN', 'GRU'.

```
    acc = accuracy_score(y_true, y_pred)
```

- `accuracy_score` — sklearn function: fraction of positions where `y_true == y_pred`.

```
    macro_f1 = f1_score(y_true, y_pred, average='macro')
```

- **f1_score** — Harmonic mean of precision/recall per class (behavior depends on average).
- **average='macro'** — Computes F1 **per class** then **averages** without weighting class frequency.

```
cm = confusion_matrix(y_true, y_pred)
```

- **confusion_matrix** — Square matrix **counts**: rows = true class, columns = predicted class (integer indices).

Printing header and report

```
print(f'\n[{prep_name} | {rep_name} | {model_name}']')
```

- **f-string** — Inserts variable values into {...} inside the string.

```
print(f'Accuracy: {acc:.4f}')
```

```
print(f'Macro F1: {macro_f1:.4f}')
```

- **{acc:.4f}** — Format float to 4 decimal places.

```
print('Classification report:')
```

```
print(classification_report(y_true, y_pred, target_names=label_encoder
```

- **classification_report** — Text table of precision/recall/F1 per class.
- **target_names=label_encoder.classes_** — Uses **string topic names** instead of numeric IDs in the report.

Confusion matrix heatmap

```
plt.figure(figsize=(6, 5))
```

- New matplotlib figure sized 6×5 inches.

```
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=label_e
```

- **sns.heatmap** — Draws matrix **cm** as colored cells.
- **annot=True** — Writes numeric count inside each cell.
- **fmt='d'** — Integer formatting for annotations.
- **cmap='Blues'** — Color palette.
- **xticklabels / yticklabels** — Label axes with actual topic names from **label_encoder.classes_**.

```
plt.title(f'Confusion Matrix - {prep_name} | {rep_name} | {model_name}')
plt.xlabel('Predicted')
```

```
plt.ylabel('Actual')
```

- Axis labels: columns = predicted, rows = true (standard confusion matrix convention).

```
plt.xticks(rotation=20)
```

```
plt.yticks(rotation=0)
```

```
plt.tight_layout()
```

```
plt.show()
```

- Rotates x labels for readability; **show()** displays plot.

Append to global results

```

all_results.append({
    'preprocessing': prep_name,
    'representation': rep_name,
    'model': model_name,
    'accuracy': acc,
    'macro_f1': macro_f1
})

```

- **append** — Adds one **dict** to **all_results** list (mutates global list defined above).
 - Keys become **columns** when converted to DataFrame later.
-

add_tuning_log

```
def add_tuning_log(prepare, rep, model, config, val_f1):
```

- **config** — Usually a **dict** of hyperparameters (TF-IDF + LR params or DNN params).

```

    tuning_logs.append({
        'preprocessing': prep,
        'representation': rep,
        'model': model,
        'config': str(config),
        'val_macro_f1': float(val_f1)
    })

```

- **str(config)** — Converts dict to readable **string** (easy CSV column).
 - **float(val_f1)** — Ensures plain Python float for serialization.
-

Mechanism of the whole cell

Initialize accumulators → define text pipeline helper → define **test** evaluation + logging → define **validation** logging.

Role and importance in the notebook

Avoids duplicating dozens of evaluation blocks; ensures every experiment logs rows into the same structures for the final comparison cell.